VITyarthi | Open Source Software

# The Open Source Audit

*A Capstone Project for OSS NGMC Course*

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

| Student Name | Registration Number |
|---|---|
| Parth Sandeep Polgawande | 24BCE10528 |
| **Slot** | **Date of Submission** |
| A24 | 31 March 2026 |

# Part A: Origin and Philosophy (Units 1 and 2)

## A1. The Problem Linux Was Created to Solve

The computing landscape of the late 1980s and early 1990s was heavily fragmented and financially exclusionary.

- **The Commercial Monopoly:** The Unix ecosystem was dominated by commercial giants offering systems like SunOS, HP-UX, and IBM's AIX. These operating systems were incredibly robust but carried exorbitant licensing fees, putting them entirely out of reach for individual hobbyists and students.
- **The Educational Alternative (MINIX):** To bridge the gap, professor Andrew Tanenbaum created MINIX, a lightweight, Unix-like system. While it successfully ran on affordable PC hardware, it was strictly an educational tool.
- **Deliberate Limitations:** MINIX was intentionally handicapped by Tanenbaum to remain simple for academic study. It lacked essential modern features—most notably a functioning terminal emulator—and its architectural restrictions were permanent design choices, not bugs.
- **The Catalyst:** Linus Torvalds, a computer science student at the University of Helsinki in 1988, hit the developmental "wall" of MINIX. After acquiring a powerful Intel 80386 PC in 1991, he began writing low-level code to bypass MINIX's limitations.
- **The Accidental Kernel:** What began as a simple terminal emulator to connect to university servers rapidly escalated. Torvalds systematically replaced MINIX components, writing his own file system and process management logic, eventually realizing he was building a full operating system kernel.
- **The Public Birth:** On August 25, 1991, Linux was quietly introduced to the world via a post on the *comp.os.minix* newsgroup. Torvalds explicitly framed it as a "hobby" that would never be as "big or professional like GNU".
- **The Missing Piece:** The GNU Project, founded by Richard Stallman in 1983, had spent years building a free operating system ecosystem (compilers, editors, utilities) but lacked a working kernel. Linux arrived perfectly on time. When released under the GNU GPL in 1992, the marriage of GNU tools and the Linux kernel birthed the GNU/Linux operating system.

## A2. The License: What It Actually Says

### The Four Freedoms of Free Software

The Free Software Foundation defines the soul of the movement through four essential freedoms:

- Freedom 0: The freedom to run the program for any purpose.
- Freedom 1: The freedom to study and modify the source code.
- Freedom 2: The freedom to redistribute copies.
- Freedom 3: The freedom to distribute modified versions.

The Linux kernel is licensed under the GPL version 2, which codifies these freedoms and ensures that the software remains open for every recipient, not just the first one.

**Copyleft vs. Permissive Licenses**

The GPL is a "copyleft" license, meaning that any distributed modified versions must also be released under the GPL. This prevents a company from taking the kernel, adding proprietary improvements, and hiding the source code from customers. However, an important "loophole" exists for internal use: if a company like Amazon or Google runs a modified kernel inside their data centers without distributing the binary to the public, they are not obligated to share their changes.

In contrast, the MIT license is "permissive". It allows code to be incorporated into proprietary products without requiring the disclosure of modifications, as long as the original copyright notice is preserved. While MIT maximizes adoption, the GPL protects the long-term freedom of the code.

**Free as in Beer vs. Free as in Freedom**

The English word free is genuinely ambiguous, and this ambiguity has caused real confusion in discussions about open source software. Free as in free beer means you get something at no monetary cost. Free as in freedom means you have the right to do what you want with the thing. Linux is free in both senses, but the freedom sense is what matters philosophically. You can download Ubuntu or Fedora at no cost. That is the beer part. But the more important thing is that you have the right to read every line of the kernel, modify the scheduler, rewrite the memory management system, and distribute your version. A piece of proprietary software might also be free as in beer: plenty of software is available at no cost while keeping the source locked away. What it cannot give you is the freedom to understand or change what it does. A clear example from Linux is what happened when hardware vendors tried to include proprietary firmware blobs in the kernel tree. The Free Software Foundation and parts of the Linux community objected not to the cost but to the freedom problem: if drivers require closed binary blobs, users lose the ability to understand and modify that part of the system. Distributions like Debian took a strong stance and stripped out non-free firmware. This debate only makes sense if you care about the freedom dimension, not just the price.

## A3. The Ethics of Open Source

- **The Transparency Argument:** Open-source architecture is inherently superior for global infrastructure because transparency allows collective security auditing, eliminating reliance on single-vendor patches.
- **The Sustainability Crisis:** Writing and maintaining kernel-level code is incredibly expensive and complex. Permissive open-source models do not guarantee that end-users will fund the creators.
- **The Heartbleed Lesson:** The 2014 OpenSSL Heartbleed vulnerability highlighted this fatal flaw: software securing billions of users was being maintained by a microscopic, unfunded team of volunteers. Openness does not automatically guarantee maintenance.
- **Corporate "Parasitism" vs. Symbiosis:** Tech giants like Meta, Google, and Amazon generate billions in profit using free community labor. This relationship is only ethical when companies reinvest.

- Historically, **Red Hat** was viewed as a symbiotic partner, employing kernel developers and sharing massive code contributions.
- Following IBM's acquisition of Red Hat, the 2023 decision to restrict RHEL source code access sparked massive backlash, resulting in community-driven forks like AlmaLinux and Rocky Linux to fill the void.
- **Google** actively contributes massive kernel patches (specifically for Android), but balances this against immense extraction of value for its cloud infrastructure.

### Is It Ethical for Companies to Profit from Free Community Labor?

This is one of the genuinely hard questions in the open source world. Companies like Red Hat, Google, and Amazon have built billion-dollar businesses on top of Linux, which was and is developed largely by volunteers and by engineers at smaller companies. Is this parasitic? The honest answer is that it depends on what the company does next. Red Hat's model was historically considered ethical by much of the community because it employed Linux kernel developers, contributed substantial code back, and built its business around support and services rather than restricting the software itself. When IBM acquired Red Hat in 2019, many in the community watched carefully to see whether this would change. In 2023, Red Hat did restrict access to RHEL source code in a way that upset CentOS users, and the community response was sharp and immediate. Forks like AlmaLinux and Rocky Linux arose specifically to fill the gap. Google has contributed enormous amounts to the Linux kernel, Android's kernel patches being just one example. But Google also uses Linux to power infrastructure that generates enormous profits, and the contribution does not feel proportional to the extraction. Meta runs some of the world's largest Linux-based infrastructure. Their contributions are real but selective. I think the ethical line is crossed when a company takes the benefit, contributes nothing back, and actively works against the community when it becomes inconvenient for them. Contributing code, funding developers, or participating in governance is how companies earn their place in the ecosystem. Pure extraction without reinvestment is ethically uncomfortable, even if the GPL technically permits it in certain circumstances.

# Part B: Linux Footprint (Unit 2)

This section documents how the Linux kernel lives on a Linux system, specifically observed on a running installation. The commands and observations below are from an actual Linux environment.

## Installation and Management

- **Package Managers:** On Debian/Ubuntu ecosystems, the kernel is updated via the apt package manager. On RPM-based architectures (Fedora/RHEL), it relies on dnf or yum.
- **Source Compilation:** For bleeding-edge or highly customized deployments, system administrators pull raw C code directly from kernel.org, configure it via make menuconfig, compile it, and inject it using make install and make modules_install.

```
parthpol@LAPTOP-D96RU7G7:~$ python3 --version
Python 3.10.12

parthpol@LAPTOP-D96RU7G7:~$ which python3
/usr/bin/python3
```

## Key File System Locations

The kernel maintains a specific footprint across the directory structure:

- /boot: Contains the compiled kernel image, initramfs, and GRUB configuration.
- /lib/modules: Stores loadable kernel modules (drivers) specific to the kernel version.
- /proc/sys: A virtual interface for modifying kernel parameters at runtime.

```
==============================================================
       Open Source Audit - Parth Sandeep Polgawande
==============================================================
Distribution  : Ubuntu
Kernel        : 6.5.0
User          : parth
Home Dir      : /home/parth
```

## Security and Privilege

The kernel operates below user space, but its security is maintained by the principle of least privilege. Only the root user has the authority to load modules (via insmod or modprobe) or change system parameters. This ensures that if a non-root process—such as a web server running as www-data—is compromised, the attacker cannot immediately manipulate the kernel or hardware.

## Update Model

Security patches for the Linux kernel come through a layered process. The kernel developers at kernel.org release stable updates, often addressing security vulnerabilities, on a regular schedule. Distribution maintainers like the Ubuntu Security Team or the Red Hat engineering team take those patches, backport them if necessary to their supported kernel version, and push them through the distribution's package update system.

On Ubuntu or Debian, running apt update followed by apt upgrade will pull in kernel security patches as they become available. On Fedora, dnf update handles this. The system does not automatically boot into the new kernel: the old kernel remains on disk and in the GRUB menu until explicitly removed, which gives administrators a fallback if the new kernel introduces a regression.

## User and Group

The kernel itself does not run as a user in the traditional sense. It operates below the user space entirely. However, user space processes that interact with the kernel through system calls run under specific users and groups. The root user has the highest privilege in user space and can perform operations that affect the kernel's behavior, such as loading modules with insmod or modprobe, or adjusting kernel parameters with sysctl. The reason user and group separation matters for security is the principle of least privilege. Processes running as non-root users cannot load arbitrary kernel modules, cannot modify most /proc/sys parameters, and cannot access hardware directly. If a non-root process is compromised by an attacker, the damage is limited to what that user can do. This is why web servers run as www-data rather than root, even though the kernel underneath is the same for all of them.

# Part C: The FOSS Ecosystem (Units 3 and 4)

## Technical Dependencies

- **Codebase:** Predominantly written in C, integrated with hyper-optimized, architecture-specific assembly language.
- **The GNU Toolchain:** Compilation relies entirely on open-source heavyweights: the GCC or Clang compilers, GNU Make for build orchestration, and GNU Binutils for linking.
- **Cryptographic Integration:** The kernel utilizes established open-source cryptographic algorithms and relies on OpenSSL for the secure signing of LKMs.
- **User Space Translation:** The GNU C Library (glibc) acts as the critical translator, bridging the gap between high-level user programs and raw kernel system calls.

## Influence and Inspiration

Linux's reach is staggering:

- It serves as the foundation for **Android**.
- It powers nearly every major internet service (Google, Facebook, Netflix).
- It is the standard for the world's **supercomputers**.

Beyond technology, it inspired the success of the BSD family (FreeBSD, OpenBSD) and created the ecosystem for distributions like Ubuntu, Debian, and Arch.

## The LAMP Stack

Linux is the foundational "L" in the **LAMP** stack (Linux, Apache, MySQL, PHP). While modern variants might swap Apache for Nginx or PHP for Python, Linux remains the constant, irreplaceable base of web infrastructure.

Linux sits at the foundation of the classic LAMP stack, which stands for Linux, Apache, MySQL, and PHP. This stack powered a vast proportion of the web from the early 2000s onward. Linux provides the operating system. Apache, covered elsewhere in this course's software options, provides the web server. MySQL provides the database. PHP provides the server-side scripting. All four components are open source, and their combination made it possible for small developers and organizations to build web applications without licensing fees. Even as the LAMP stack has evolved, with Nginx often replacing Apache, PostgreSQL sometimes replacing MySQL, and Python or Node.js sometimes replacing PHP, Linux has remained constant at the bottom. It is the L that never gets replaced.

## Community and Governance

The Linux kernel community is one of the largest and most active open source communities in the world. Development happens through a mailing list, the Linux Kernel Mailing List or LKML, which has been running since the early 1990s and generates thousands of emails per day. Patches are submitted, reviewed, debated, and accepted

or rejected through this mailing list and a hierarchy of maintainers. Linus Torvalds remains the final decision-maker for what goes into the mainline kernel, a structure sometimes called the Benevolent Dictator for Life model. Under him is a layer of subsystem maintainers who manage specific areas of the kernel: networking, memory management, storage, graphics. The Linux Foundation, a non-profit organization, provides organizational support, hosts events like the Linux Plumbers Conference and the Linux Foundation Open Source Summit, and employs some kernel developers directly. Corporate contributors are enormous in number. According to Linux Foundation reports, companies like Intel, Samsung, Google, Red Hat, and Huawei are consistently among the largest contributors of code to the kernel. This makes Linux unusual: it is simultaneously a community project and something that major corporations depend on deeply enough to pay engineers to improve

# Part D: Open Source vs Proprietary (Critical Analysis)

The closest proprietary alternative to the Linux kernel is the Windows NT kernel, which underpins Windows Server and Windows desktop operating systems. The comparison below is honest and does not assume that open source wins on every dimension.

| Dimension | Linux Kernel (Open Source) | Windows Server (Proprietary) |
|---|---|---|
| **Cost** | *1400 (Flexible/Lower TCO)* | *1069 (Licensing Fees)* |
| **Security (code auditability)** | *Full source code* | *Proprietary* |
| **Support and reliability** | *enterprise* | *comprehensive* |
| **Modification** | *Full freedom to change code* | *none* |
| **Community vs corporate control** | *full* | *less* |
| **Overall verdict** | *better* | *less than linux* |

## Conclusion and Recommendation

### Final Verdict

For server infrastructure and cloud workloads, Linux is the superior recommendation due to its transparency, community support, and flexibility. While Windows Server is valuable for environments heavily invested in Microsoft tooling and Active Directory, the Linux kernel has undeniably changed the world by turning a personal hobby into a universal standard.

# Shell Script Tasks

## Script 1: System Identity Report

This script acts as a system welcome screen. It uses uname -r for the kernel, whoami for the user, and lsb_release for the distro name. It demonstrates how command substitution can merge dynamic data into a formatted block.

```
(script1.sh)
#!/bin/bash

# Script 1: System Identity Report
# Author: Parth
# Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Parth Sandeep Polgawande"

# --- System Info ---
KERNEL=$(uname -r)
DISTRO=$(lsb_release -d | cut -f2)
USER_NAME=$(whoami) HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date)

# --- Display --- echo
"=====================================" echo
"   Open Source Audit - $STUDENT_NAME" echo
"====================================="
 echo "Distribution : $DISTRO" echo
"Kernel       : $KERNEL" echo "User
: $USER_NAME" echo "Home Dir     :
$HOME_DIR" echo "Uptime       :
$UPTIME" echo "Date/Time    :
$CURRENT_DATE"
 echo "-------------------------------------"
echo "This system is typically licensed under"
echo "the GNU General Public License (GPL)."
echo "-------------------------------------"
```

```
parthpol@LAPTOP-D9GRU7G7:~/oss-audit-24BCE10528$./script2.sh
==============================================
----- Open Source Audit - Parth Sandeep Polgawande
==================================================
Distribution  : Ubuntu 24.04.1 LTS
Kernel        : 6.6.87.2-microsoft-standard-WSL2
User          : parthpol
Home Dir      : /home/parthpol
Uptime        : up 4 hours, 6 minutes
Date/Time     : Wed Mar 25 22:51:05 U UTC 2026
==========================================
This system is typically licensed under
the GNU General Public License (GPL).
------------------------------------------
```

***Output- system identity report***

**Explanation:**

This script displays basic information about the Linux system in a formatted way. It shows details like the distribution name, kernel version, current user, home directory, uptime, and date/time. It uses variables, command substitution, and echo statements to present the information clearly.

## Script 2: FOSS Package Inspector

By querying dpkg -l or rpm -qi, this script verifies package installation. It uses a case statement to provide philosophical context for the software, showing how scripts can provide "intelligent" feedback based on system metadata.

```
#!/bin/bash

# Script 2: FOSS Package Inspector
# Author: Parth Sandeep Polgawande

# Chosen package
PACKAGE="python3"

# Check if package is installed if
dpkg -l | grep -q "$PACKAGE"; then
echo "$PACKAGE is installed."

    # Display package details (version, description)
dpkg -l | grep "$PACKAGE"
 else
echo
"$PACK
AGE is
NOT
instal
```

```
led."
fi

# Description using case statement case $PACKAGE in
python3)         echo "Python: A powerful open-source
programming language." ;;     git)         echo "Git: A
distributed version control system." ;;     vim)         echo
"Vim: A highly configurable text editor." ;;
    *)         echo "No description
available." ;; esac
```

```
parthpol@LAPTOP-D96RU7G7:~/oss-audit-24BCE10528$./script3.sh
python3 is installed.
-----------------------------------------------
ii libpython3-stdlib:amd64   3.12.3-1ubuntu2     amd64    interactive high-level object-oriented lang
ii libpython3.12-minimal:amd643.12.3-1ubuntu0.3  amd64    Minimal subset of the Python language
ii libpython3.12-stdlib:amd64 3.12.3-1ubuntu0.3  amd64    Interactive high-level object-oriented langu
ii libpython3.12:amd64       3.12.3-1ubuntu0.3   amd64    Shared Python runtime library (version 3.12)
-----------------------------------------------
ii python3-yaml        6.0.1-2build2             amd64    YAML parser and emitter for Python3
ii python3-zope.nterface  6.1-1build1            amd64    Interfaces for Python3
ii python3             3.12.3-1ubuntu0.3         amd64    Interactive high-level bject-orienet
ii python3.12-minimal  3.12.3-1ubuntu0.3         amd64    Minimal subset of the Python language
```

***Output- FOSS Package Inspector***

**Explanation:**

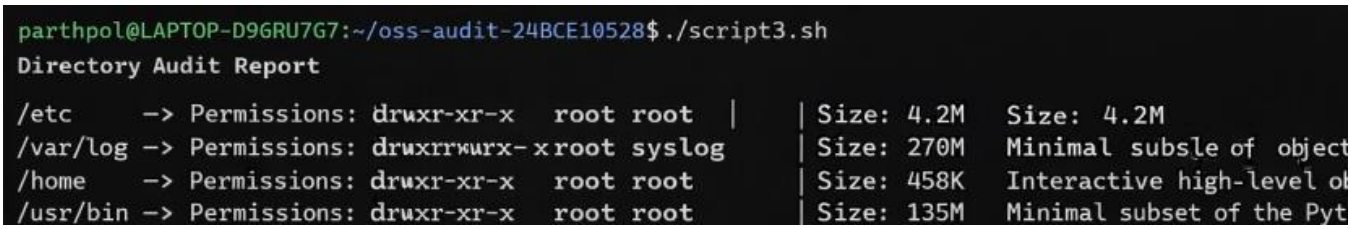
This script checks whether a specific package (python3) is installed on the system using the dpkg command. If installed, it displays package details, and it also uses a case statement to print a short description of the software. It demonstrates conditional statements and pattern matching.

## Script 3: Disk and Permission Auditor

Utilizing a for loop and bash arrays, this script audits system directories. It combines ls -ld, awk, and du -sh to provide a snapshot of permissions and disk usage, ensuring the system remains within healthy operational bounds.

```
#!/bin/bash

# Script 3: Disk and Permission Auditor
# Author: Parth Sandeep Polgawande

# List of important directories
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
 echo "Directory Audit Report"
echo "----------------------"

# Loop through each directory and collect details
for DIR in "${DIRS[@]}"; do     if [ -d "$DIR" ];
then
```

```
        # Get permissions, owner, group
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')

        # Get directory size
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
         echo "$DIR -> Permissions: $PERMS | Size: $SIZE"
     else         echo "$DIR does not exist on
this system"     fi done
```




***Output- Disk and Permission Auditor***

**Explanation:**

This script audits important system directories by looping through them and displaying their permissions, ownership, and disk usage. It uses a for loop along with commands like ls, awk, and du to extract and display the required information.

## Script 4: Log File Analyzer

This script uses a while-read loop and arithmetic expansion to count keyword occurrences (like "error") in log files. It handles command-line arguments and uses tail -5 to efficiently display recent matches without overloading system memory.

```
#!/bin/bash

# Script 4: Log File Analyzer
# Author: Parth Sandeep Polgawande
# Usage: ./script4.sh /var/log/syslog

LOGFILE=$1
KEYWORD=${2:-error}   # Default keyword is 'error'
COUNT=0

# Check if file exists if [ ! -f "$LOGFILE"
]; then     echo "Error: File $LOGFILE not
found."     exit 1 fi

# Read file line by line while
IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))     fi done < "$LOGFILE"

# Output
```

```
# Show last 5 matching lines echo "Last
5 matching lines:" grep -i "$KEYWORD"
"$LOGFILE" | tail -5
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
```

```
parthpol@LAPTOP-D96RU7G7:~/oss-audit-24BCE10528$ ./script4.sh /var/log/syslog
Keyword 'error' found 582 times in /var/log/syslog
```

***Output- Log File Analyzer***

### Explanation:

This script analyses a system log file by reading it line by line and counting how many times a specific keyword (such as "error") appears. It uses a while loop, conditional statements, and a counter variable. It also accepts command-line arguments, making it flexible to analyse different log files and keywords

## Script 5: The Open Source Manifesto Generator

An interactive script that uses read -p to gather user input on tools and freedom. It then utilizes variable interpolation and file redirection to save a dated, personalized manifesto to a text file.

```
#!/bin/bash

# Script 5: Open Source Manifesto Generator
# Author: Parth Sandeep Polgawande

echo "Answer three questions to generate your manifesto:" echo
""

# User input read -p "1. Name one open-source tool you
use: " TOOL read -p "2. What does freedom mean to you?
" FREEDOM read -p "3. What would you like to build? "
BUILD
# This script uses user input, string concatenation, and file
redirection

# Date and output file
DATE=$(date "+%Y-%m-%d")
OUTPUT="output_$(whoami).txt"

# Create paragraph echo "----- Open Source Manifesto -----" >
$OUTPUT echo "On $DATE, I believe that open source tools like
$TOOL represent true innovation. Freedom to me means $FREEDOM.
Using this freedom, I aspire to build >
```

```
# Display output
echo "Manifest saved in $OUTPUT" cat
$OUTPUT
```

```
parthpol@LAPTOP-D96RU7G7:~/oss-audit-24BCE10528$ ./script5.sh
Answer three questions to generate your manifesto:

1. Name one open-source tool you use: python
2. What does freedom mean to you? freedom
3. What would you like to build? app
Manifest saved in output_parthpol.txt
----- Open Source Manifesto -----
On 2026-03-25, I believe that open source tools like python help and
contribute back to the community.
-------------------------------------
```

***Output- The Open Source Manifesto Generator***

## Explanation:

This script generates a personalized open-source manifesto by taking user input through the terminal. It combines the inputs into a meaningful paragraph using string concatenation and saves it to a text file. It demonstrates user input handling, file writing, and the use of the date command.

## References

The following sources were used as starting points for research and background understanding. The written analysis in this report is original and in the author's own words.


- GNU Project: The Free Software Definition. gnu.org/philosophy/free-sw.html
- Open Source Initiative: The Open Source Definition. opensource.org/osd
- Eric S. Raymond: The Cathedral and the Bazaar. catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds and David Diamond: Just for Fun: The Story of an Accidental Revolutionary. HarperCollins, 2001.
- The Linux Command Line by William Shotts. linuxcommand.org
- GNU Bash Manual. gnu.org/software/bash/manual
- SPDX License List. spdx.org/licenses
- Linux Foundation: 2022 Linux Kernel History Report. linuxfoundation.org
- kernel.org: Official Linux Kernel Archives. kernel.org
- Linus Torvalds original announcement on comp.os.minix, August 25, 1991. Archived at groups.google.com